DOCUMENTACIÓN TÉCNICA Y FUNCIONAL

Ropa hospitalaria y residuos

Arquitectura, procesos, permisos y operación del sistema

Sistema de Registro y Control de Ropa Hospitalaria y Residuos

Elaborado por
Eval Samuel Molina Fuertes

Pasto, Nariño · 11 de noviembre de 2026 · Versión documental 1.0

Contenido y alcance

Contenido

3. Visión general y alcance funcional
4. Módulos y operación
5. Trazabilidad y reglas de negocio
6. Arquitectura y componentes
7. Modelo de datos
8. Roles y permisos
9. Seguridad implementada
10. API e integración
11. Instalación y despliegue
12. Respaldos y continuidad
13. Verificación, límites y fuentes

Identificación documental

Autor: **Eval Samuel Molina Fuertes**
Fecha: 11 de noviembre de 2026
Versión documental: 1.0
Aplicación: Sistema de Registro y Control de Ropa Hospitalaria y Residuos
Institución indicada en el proyecto: Clínica Cardioneurovascular Pabón S.A.S., Pasto, Nariño.

Criterio de elaboración

Se describe la implementación disponible en el repositorio local. Las afirmaciones sobre configuración corresponden al código y a los valores predeterminados revisados; no certifican la configuración de un servidor en producción.

Documentos de referencia

Los PDF de Equipos de Emergencia aportan la separación entre tutorial y documentación técnica. Sus módulos, tecnologías, autores y estados de despliegue pertenecen a otra aplicación y no se trasladan a este sistema.

Visión general y alcance funcional

Propósito

Centralizar el registro y control de ropa hospitalaria y residuos, vinculando las operaciones con sede, servicio, fecha, jornada y responsables. Facilita revisar los movimientos, documentar novedades y producir información para seguimiento y facturación.

Ropa hospitalaria

El proceso incluye entrega de ropa sucia, recepción de ropa limpia desde lavandería y distribución a los servicios. Incorpora detalle por prenda, rótulos, seguimiento del ciclo de retorno, validación de entrega a lavandería y corte de control.

Residuos

El proceso comprende generación y recolección, categorías y tipos de residuo, pesos por tipo cuando corresponde, consolidado de peligrosos y asociación de entregas con gestores externos para facturación y conciliación.

Administración y seguimiento

Dispone de catálogos, cuentas y perfiles, parámetros de jornadas y umbrales, novedades, revisión del día anterior, consolidados y exportaciones a Excel. El RH1 se construye con columnas configurables.

Límite del alcance

La revisión es documental y de código. No se verificaron disponibilidad del servidor, datos productivos, configuración institucional definitiva del RH1 ni la ejecución de respaldos en infraestructura real.

Módulos y operación

Acceso y panel

La interfaz solicita usuario y contraseña. Tras autenticarse, muestra opciones según el perfil. El panel y la revisión del día anterior limitan las listas operativas a movimientos en los que participó la persona; administración tiene alcance institucional.

Captura de ropa

La entrega de ropa sucia identifica sede y servicio. Personal de servicio captura prendas y cantidades sin peso; el operario captura el pesaje o completa la entrega que le asignaron. La recepción limpia registra peso total, tara y datos del retorno; la distribución registra prenda y cantidad de unidades por servicio receptor.

Captura de residuos

Personal de servicio marca tipos y asigna un receptor sin pesar. El operario puede registrar grupo, categoría, tipo específico y pesaje; también completa los pesos de entregas asignadas. Los grupos definidos son No peligroso, Riesgo biológico, Otro peligroso y Otros.

Control y reportes

Rótulos conserva identificación y señala entregas sin rotular. Validación compara peso declarado y calculado. Consolidados agrupa por sede, servicio, categoría y jornada. RH1 y facturación reúne el formato ambiental, valores facturados y conciliación con el gestor.

Gestores y novedades

Una entrega al gestor enlaza una recolección con un gestor, kg facturados, valor y número de factura opcional. Las novedades describen situaciones como faltantes, deterioro, derrames o diferencias de peso y conservan su relación con el movimiento.

Trazabilidad y reglas de negocio

Identidad de la operación

Movimiento conserva tipo, fecha, hora, jornada, sede, servicio cuando aplica, quién entrega, quién recibe, creador, fecha de creación, estado y periodo de facturación. Los modelos principales usan historial de cambios mediante django-simple-history.

Pesaje

Peso neto = peso total − tara. El modelo de pesaje y los formularios validan los datos; la tara no puede superar el total. Una entrega pendiente de pesaje no se interpreta como cero. El operario asignado como receptor puede registrar su peso una vez; los errores posteriores se tramitan como corrección.

Correcciones y novedades

Para cuentas operativas, la corrección exige que el registro sea propio y esté dentro de la ventana configurada, de 60 minutos por defecto. La Administradora y la cuenta técnica superusuario no tienen ese límite. Quien entrega o recibe puede reportar una novedad; administración también puede hacerlo.

Ciclo y cortes

La recepción de ropa limpia puede enlazarse con la entrega de ropa sucia mediante mov_origen. El corte de control combina la entrega de las 18:00 del día anterior con la de las 10:00 del día actual. Es una consulta de seguimiento y no se suma otra vez al consolidado del periodo.

Conformidad y conservación

La evaluación compara peso declarado y peso del sistema, con umbrales en kg y porcentaje. Ambos umbrales parten de cero y el bloqueo está desactivado por defecto. La aplicación incorpora controles para impedir eliminación de registros; las correcciones deben conservar trazabilidad.

Arquitectura y componentes

Vista de componentes

Interfaz React → API REST Django → PostgreSQL.
La configuración de producción incluida usa Caddy → Gunicorn/Django → PostgreSQL. Un servicio independiente ejecuta respaldos. El proyecto también conserva vistas y plantillas Django.

Frontend

React 19, TypeScript y Vite 8. React Router administra rutas; TanStack Query consulta y actualiza datos; React Hook Form gestiona formularios y Axios realiza peticiones HTTP. El código organiza pantallas por dominio: ropa, residuos, movimientos, reportes y catálogos.

Backend

Python 3.12 en la imagen Docker, Django 5.2 y Django REST Framework 3.16. La API se publica bajo /api/. Las áreas principales son core, ropa, residuos, movimientos y reportes. Las operaciones reutilizan validaciones de formularios y servicios del backend.

Persistencia y exportación

PostgreSQL 16 según Docker Compose. psycopg conecta la aplicación con la base. openpyxl genera exportaciones Excel; django-simple-history conserva cambios. django-constance almacena parámetros editables en la base de datos.

Producción definida

Docker Compose declara db, web, caddy y respaldo, con volúmenes persistentes. Caddy publica 80/443 y sirve /static/. La configuración actual redirige el resto de solicitudes a Django; no incluye la publicación del build de React. Integrar ese build requiere configurar su servicio estático y el fallback de rutas.

Modelo de datos

Estructura institucional

Sede tiene varias AreaServicio. El servicio indica si genera ropa y/o residuos. Color y AreaColor describen asociaciones informativas; el color no identifica de forma única al servicio. Usuario extiende AbstractUser y añade documento, rol y estado activo.

Núcleo transaccional

Movimiento es la entidad central. Se relaciona con sede, servicio de origen, usuarios de entrega y recepción, y uno o varios Pesaje. También contiene firmas dibujadas y una referencia opcional a otro movimiento para el ciclo de ropa. Novedad pertenece a un movimiento.

Ropa

Prenda define el catálogo. DetalleRopa asocia prendas y cantidades con el movimiento. Rotulo registra identificación y contenido de una entrega. ValidacionEntrega conserva la comparación de peso para sede, fecha y jornada.

Residuos y facturación

CategoriaResiduo organiza grupos y jerarquía de categorías. DetalleResiduo relaciona el movimiento con categorías y peso cuando aplica. EntregaGestor tiene relación uno a uno con Movimiento y referencia a GestorExterno. ColumnaRH1 define el orden, nombre y categorías o grupo que alimentan el reporte.

Configuración y protección

ConfiguracionJornada define horarios por sede y proceso. Los parámetros generales se almacenan mediante Constance. Las relaciones sensibles utilizan protección referencial y existen restricciones de unicidad en entidades como categorías por movimiento y servicios por sede.

Roles y permisos

Personal de servicio · SERVICIO

Registra entregas de ropa por conteo y tipos de residuos sin pesar. Puede distribuir ropa limpia por unidades. Asigna al operario que recibe y completa el pesaje. Consulta movimientos en los que participa y reporta novedades cuando entrega o recibe.

Usuario · USUARIO

Es el perfil de operación y pesaje. Dispone de recepción de ropa limpia, rótulos, validación y consolidado de peligrosos, además de los flujos de captura habilitados. Solo el receptor operativo asignado puede pesar una entrega pendiente.

Administradora · ADMIN

Gestiona cuentas, catálogos, configuración, novedades y reportes; registra entrega al gestor y consulta RH1/facturación. Corrige registros sin límite de ventana. La navegación separa sus tareas de gestión de la captura operativa.

Cuenta técnica

El superusuario tiene excepciones explícitas en permisos del backend para soporte y administración. No constituye un cuarto perfil operativo del catálogo. Su contraseña solo la modifica otro superusuario según la documentación de despliegue.

Aplicación de permisos

El frontend usa rutas protegidas y el backend comprueba autenticación y permisos por endpoint mediante IsAdministradora, IsOperario e IsUsuario y controles equivalentes. Las firmas dibujadas solo se muestran a participantes del movimiento y administración. Los cortes y consolidados tienen alcance institucional por diseño.

Seguridad implementada

Autenticación y sesión

La API usa sesión Django mediante cookies y protección CSRF. El flujo obtiene primero la cookie CSRF y luego inicia sesión. La configuración establece 1.800 segundos de sesión y renovación por actividad; la sesión expira al cerrar el navegador según la cookie configurada.

Credenciales y bloqueo

Django valida contraseñas con mínimo de 10 caracteres, control de similitud, claves comunes y claves solo numéricas. django-axes bloquea una cuenta después de cinco fallos durante 15 minutos y registra intentos con IP. La Administradora asigna y cambia las contraseñas.

Transporte y exposición

La configuración de producción activa cookies seguras y cabeceras de seguridad. Caddy obtiene HTTPS al configurar un dominio real y restringe /admin/ a redes permitidas. PostgreSQL no publica un puerto del host en el compose de producción. Estos controles dependen del despliegue efectivo.

Datos y auditoría

El historial de modelos permite revisar cambios. Los permisos limitan captura, corrección y acceso a firmas. Las cuentas operativas ven sus participaciones en panel y revisión del día anterior. Los secretos se cargan desde variables de entorno, que no deben incluirse en documentación ni repositorios.

Comprobación operativa

Ejecutar python manage.py check --deploy en el entorno objetivo. Revisar CORS y CSRF para los orígenes reales, redes de administración, HTTPS y advertencias de respaldos. Si existe un proxy adicional, configurar adecuadamente la confianza y la IP del cliente antes de depender de las restricciones de red.

API e integración

Base y autenticación

Prefijo: /api/.
GET auth/csrf/ obtiene la cookie CSRF.
POST auth/login/ inicia sesión con X-CSRFToken.
GET auth/me/ consulta el usuario autenticado.
POST auth/logout/ cierra sesión.
auth/cambiar-clave/ atiende el cambio autorizado.

Operaciones principales

movimientos/entrega-ropa-sucia/
movimientos/recepcion-ropa-limpia/
movimientos/distribucion-ropa-limpia/
movimientos/generacion-residuo/
movimientos/recoleccion-residuo/
Los endpoints de creación usan validaciones específicas para cada proceso.

Catálogos y control

catalogos/sedes/, catalogos/servicios/, catalogos/prendas/, catalogos/categorias-residuo/, catalogos/gestores-externos/ y catalogos/columnas-rh1/. El router también publica usuarios/, movimientos/, novedades/, rotulos/ y validacion-entrega/, según los permisos y acciones definidos.

Consultas y exportaciones

ropa/ciclo-retorno/, ropa/corte-control/ y residuos/corte-peligrosos/. Reportes en consolidados/<clave>/, rh1/, facturacion/resumen/ y facturacion/conciliacion/. Hay rutas .xlsx para consolidados, novedades, RH1 y facturación.

Configuración del cliente

VITE_API_URL usa http://localhost:8000/api como valor de desarrollo. Si frontend y API tienen distinto origen, configurar CORS_ALLOWED_ORIGINS y CSRF_TRUSTED_ORIGINS. Las solicitudes deben enviar credenciales de sesión y, en escrituras, el token CSRF. Consultar serializers y vistas para el contrato exacto de cada operación.

Instalación y despliegue

Desarrollo local

Backend: crear un entorno Python, instalar backend/requirements.txt, configurar PostgreSQL y variables del entorno local, aplicar migraciones y ejecutar python manage.py runserver desde backend/.
Frontend: instalar dependencias en frontend/ con npm install, configurar VITE_API_URL y ejecutar npm run dev. Puertos de referencia: API 8000 e interfaz 5173.

Preparación de producción

Usar Linux con Docker y Docker Compose. Copiar backend/.env.example como .env y definir valores reales de clave de Django, hosts y acceso a PostgreSQL. Configurar el dominio en Caddyfile, las redes de administración y los parámetros de respaldo. No usar los valores de ejemplo como credenciales productivas.

Arranque documentado

Desde backend/:
docker compose -f docker-compose.prod.yml up -d --build
El entrypoint del contenedor aplica la preparación de Django. Crear la cuenta técnica mediante el comando de administración correspondiente y revisar los registros de los servicios.

Publicación de React

npm run build en frontend/ produce los archivos de distribución. El compose y Caddyfile revisados publican Django y sus archivos estáticos. Para exponer la interfaz React debe definirse dónde servir su build, mantener /api/ hacia Django y resolver las rutas de la SPA.

Verificación posterior

Ejecutar check --deploy, confirmar HTTPS, acceso por rol, cookies y CSRF, creación y consulta de un movimiento de prueba y generación de un Excel. Verificar persistencia de datos y respaldos. La documentación del repositorio contiene el detalle operativo; aquí no se certifica un despliegue ejecutado.

Respaldos y continuidad

Programación y retención

El servicio respaldo está definido para ejecutar respaldo_diario al arrancar y cada 24 horas. El manual establece retención de 35 copias locales en backups_data. Como alternativa, puede programarse el comando con cron para una hora fija.

Protección y copia externa

La implementación admite cifrado GPG con llave pública, copia a un almacenamiento externo montado y correo ante fallos. La llave privada debe mantenerse en custodia fuera del servidor. La configuración de copia externa conserva las últimas 35 copias y la última de cada uno de los 12 meses previos, según el manual.

Control periódico

Comprobar el resultado del servicio, consultar listbackups y confirmar existencia de copias externas. check --deploy advierte con core.W001, core.W002 y core.W003 si faltan componentes de protección del respaldo. La configuración por sí sola no demuestra que las copias sean recuperables.

Restauración

Procedimiento documentado: identificar la copia, detener web y caddy, restaurar con dbrestore y volver a levantar los servicios. dbrestore -i <archivo> permite seleccionar una copia. Los respaldos cifrados requieren la llave privada y la opción --decrypt, con custodia temporal controlada.

Prueba de recuperación

Realizar una restauración trimestral en una base separada y registrar archivo, fecha, duración y conteos verificados. El manual del repositorio refiere una prueba previa; esta elaboración documental no ejecutó una restauración ni validó los respaldos de un servidor real.

Verificación, límites y fuentes

Estado comprobado

Se identificaron implementaciones de pantallas y API para ropa, residuos, movimientos, reportes y catálogos. El README del frontend aún describe varias pantallas como placeholders; esa nota está desactualizada frente a los archivos revisados. La existencia del código no sustituye una prueba funcional de todas las rutas.

Validación recomendada para aceptación

Probar acceso de los tres perfiles; conteo sin peso y pesaje posterior por el receptor; ropa limpia y distribución; clasificación de residuos; permisos y ventana de corrección; novedades; reportes y Excel; conciliación con el gestor; bloqueo de acceso y recuperación de respaldos. Usar datos de prueba controlados.

Pendientes de confirmación institucional

Confirmar columnas y presentación definitiva del RH1, sedes y servicios activos, jornadas, umbrales, gestores y tarifas, responsables de administración, dominio, integración del build React y evidencias de operación y continuidad en el servidor objetivo.

Fuentes del proyecto

README.md; backend/DESPLIEGUE.md; backend/requirements.txt; backend/config/settings/; backend/config/api_urls.py; backend/docker-compose.prod.yml; backend/Caddyfile; modelos, permisos y servicios de core, movimientos, ropa y residuos; frontend/package.json; frontend/src/App.tsx y pantallas de frontend/src/features/.

Referencias de presentación

Tutorial_Sistema_Equipos_Emergencia.pdf y Documentacion_Sistema_Equipos_Emergencia.pdf, aportados por el usuario, se usaron como referencias de estructura documental.

Elaborado por: Eval Samuel Molina Fuertes
11 de noviembre de 2026 · Versión documental 1.0